

Contents

xAI Grok Coding Tools: Comprehensive Architecture Document	2
Table of Contents	2
1. Overview — Current State	2
2. Architecture — How It All Fits Together	3
High-Level Architecture	3
API Architecture	3
SDK Architecture	4
3. Core Products for Coding	4
3.1 Grok Studio (GA — April 2025)	4
3.2 Grok Build (Waitlist — January 2026)	4
3.3 Community Grok CLI (Third-Party — superagent-ai/grok-cli)	4
4. Models — The Engine Room	5
Current Model Lineup (March 2026)	5
Key Coding Model: grok-code-fast-1	5
Grok 4.20 Multi-Agent System	5
Open Source Models	6
5. Tool System — Function Calling and Built-in Tools	6
Built-in Server-Side Tools	6
Code Execution Sandbox	6
Client-Side Function Calling	6
Collections API (RAG for Codebases)	7
6. API Access — Using Grok for Coding	7
Getting Started	7
Two API Endpoints	7
Official Python SDK	8
Third-Party SDK Integrations	8
Batch API	8
7. IDE Integration — Editor Plugins	8
Official Support (via OpenAI-compatible API)	8
Cursor Setup	8
Cline Setup	9
Community VS Code Extensions	9
GitHub Copilot Chat	9
8. Agent Capabilities — Autonomous Coding	9
Current Autonomous Capabilities	9
What Is Missing vs. Claude Code	9
9. Unique Features — What Grok Does Differently	10
9.1 Real-Time X/Twitter Data	10
9.2 2-Million Token Context Window	10
9.3 Multi-Agent Architecture (Grok 4.20)	10
9.4 Arena Mode (Grok Build — Coming)	10
9.5 Collections API for Codebase RAG	10
9.6 Aggressive Pricing	10
9.7 Speed	10
10. Limitations — Honest Assessment	10
10.1 No Shipping CLI Agent (Yet)	10
10.2 Code Quality Consistency	10
10.3 No Vision for Code (Partial)	11
10.4 No Git Integration	11
10.5 No Project Context System	11
10.6 No Hook System	11
10.7 No MCP Integration	11
10.8 Ecosystem Fragmentation	11
10.9 Speed vs. Depth Trade-off	11
11. Roadmap — What Is Coming	11

Confirmed / In Development	11
Grok 5 Details	12
Grok 4.20 Evolution	12
12. Comparison Matrix	12
Grok vs. Claude Code vs. Codex CLI (March 2026)	12
Bottom Line	12
Appendix: Key URLs	12

xAI Grok Coding Tools: Comprehensive Architecture Document

Last Updated: March 25, 2026 **Maturity Assessment:** Rapidly evolving but less mature than Claude Code or Codex CLI as integrated coding platforms. Grok’s coding story is fragmented across multiple products (Grok Studio, Grok Build, grok-code-fast-1, API tools) rather than a single unified CLI experience.

Table of Contents

1. Overview — Current State
2. Architecture — How It All Fits Together
3. Core Products for Coding
4. Models — The Engine Room
5. Tool System — Function Calling and Built-in Tools
6. API Access — Using Grok for Coding
7. IDE Integration — Editor Plugins
8. Agent Capabilities — Autonomous Coding
9. Unique Features — What Grok Does Differently
10. Limitations — Honest Assessment
11. Roadmap — What Is Coming
12. Comparison Matrix

1. Overview — Current State

xAI does **not** have a single, polished CLI coding agent equivalent to Claude Code or OpenAI Codex CLI. Instead, Grok’s coding capabilities are distributed across several products and surfaces:

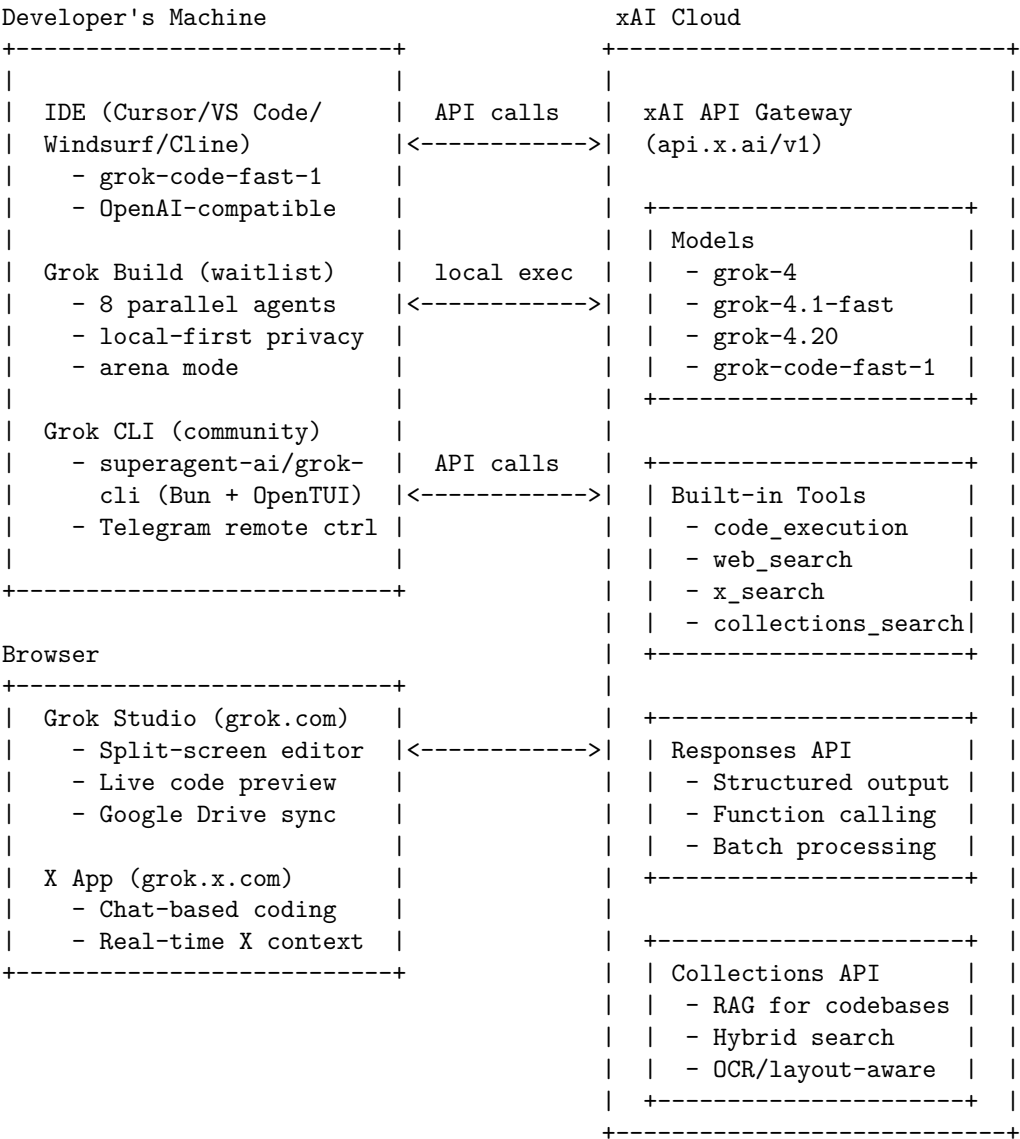
Product	Type	Status (March 2026)
Grok Studio	Web-based collaborative workspace	GA (free + premium)
Grok Build	Local-first CLI coding agent	Waitlist (announced Jan 2026)
grok-code-fast-1	API model optimized for coding	GA via API
Grok 4.20 Multi-Agent	4-agent system (web UI)	Beta
xAI API	REST/gRPC API with tool calling	GA
Grok CLI (community)	Open-source terminal agent	Third-party (superagent-ai/grok-cli)

The key takeaway: xAI has strong API-level coding capabilities and a promising but unreleased CLI agent (Grok Build). Their coding model (grok-code-fast-1) is competitive on benchmarks and widely integrated into third-party

editors. But the integrated, end-to-end “coding agent in your terminal” experience that Claude Code and Codex CLI offer is still in development.

2. Architecture — How It All Fits Together

High-Level Architecture



API Architecture

xAI provides two primary API interfaces:

1. **Chat Completions API** (/v1/chat/completions) — OpenAI-compatible endpoint. Drop-in replacement for any tool that supports custom OpenAI base URLs.
2. **Responses API** (/v1/responses) — Newer endpoint supporting server-side tools (web_search, x_search, code_interpreter, collections_search) and client-side function calling.

The API is OpenAI-compatible by design, which is why grok-code-fast-1 works seamlessly in Cursor, Cline, Windsurf, and GitHub Copilot without custom plugins.

SDK Architecture

The official Python SDK (`xai-sdk`) uses gRPC under the hood (not REST), offering:

- Synchronous client (`xai_sdk.Client`) and async client (`xai_sdk.AsyncClient`)
- Built-in retry with exponential backoff on UNAVAILABLE errors
- gRPC status code-based error handling
- Raw proto access via `.proto` attribute on responses
- Python 3.10+ required

3. Core Products for Coding

3.1 Grok Studio (GA — April 2025)

What it is: A web-based split-screen collaborative workspace at `grok.com` for writing code alongside AI assistance.

Key Features:

- Split-screen interface: chat on one side, code/preview on the other
- Live code execution in a sandboxed environment
- Supports Python, JavaScript, TypeScript, C++, Bash
- Real-time preview for web apps and browser games (Phaser.js support)
- Google Drive integration (import/edit/sync Docs, Sheets, Slides)
- Multi-format document support (essays, reports, code, data)
- Available to both free and premium users

Architecture: Entirely browser-based. Code runs in xAI's sandboxed environment, not locally. Think of it as a lighter competitor to Replit or `v0.dev`, not a CLI tool.

Limitations: Not a local development tool. No terminal access. No Git integration. No project-level codebase understanding. Best for prototyping and one-off scripts.

3.2 Grok Build (Waitlist — January 2026)

What it is: xAI's forthcoming local-first CLI coding agent. The most direct competitor to Claude Code and Codex CLI.

Key Features:

- **Local-first architecture:** All code executes on the developer's machine. No source code transmitted to xAI servers.
- **8 parallel AI agents:** Simultaneously plan, search documentation, and write code on a single project. Outputs appear side by side with a context usage tracker.
- **Arena Mode** (in development): Automated evaluation layer where agents compete or collaborate. Outputs ranked algorithmically before human review. Identified in code traces as of February 2026 but not yet publicly available.
- **Natural language to code:** Describe what you want; the agent handles planning, code generation, and execution.
- **Fine-grained permissions:** Controls over file access, script execution, and network requests.
- **Auditable actions:** Every agent action is visible before execution.

Features in Active Development (from code analysis):

- Dictation support
- Navigation tabs (Edits, Files, Plans, Search, Web Page views)
- Live code previews
- GitHub integration
- Share and Comments (collaboration)

Status: Announced January 12, 2026. Public waitlist at `grokai.build`. As of March 2026, still in waitlist/early access phase. Not generally available.

Underlying Model: `grok-code-fast-1` (70.8% SWE-Bench Verified)

How It Differs From Claude Code:

- Claude Code, Codex CLI, and Gemini CLI are fundamentally single-agent tools (though Claude Code has background agents). Grok Build's 8 concurrent agents working in parallel is its core architectural differentiator.
- Arena Mode (agents competing and being ranked) has no direct equivalent in Claude Code or Codex.

3.3 Community Grok CLI (Third-Party — `superagent-ai/grok-cli`)

What it is: An open-source terminal agent built by the community (not xAI). Brings Grok to the terminal.

Repository: `github.com/superagent-ai/grok-cli`

Key Features:

- Built with Bun and OpenTUI
- Supports `grok-code-fast-1`, `grok-4-1-fast-reasoning`, `grok-4.20-multi-agent` models
- Real-time X search (`search_x`) and web search (`search_web`) tools
- Built-in `generate_image` and `generate_video` tools
- Sub-agents enabled by default
- Telegram remote control: pair via `/remote-control`, DM your bot to control terminal sessions
- Named agent definitions in `~/.grok/user-settings.json`

Prerequisites: Node 18+, xAI API key, modern terminal emulator

Status: The upstream repo has had maintenance issues. A community fork supports the modern xAI Responses API. Not an official xAI product.

4. Models — The Engine Room

Current Model Lineup (March 2026)

Model	Context Window	Input \$/M tokens	Output \$/M tokens	Cached Input \$/M	Best For
grok-4.20	256K	~\$3.00	~\$15.00	—	Flagship, multi-agent system
grok-4	256K	\$3.00	\$15.00	—	Complex reasoning
grok-4.1-fast	2,000,000 (2M)	\$0.20	\$0.50	—	Fast inference, huge context
grok-4-fast-reasoning	2,000,000 (2M)	\$0.20	\$0.50	—	Fast reasoning tasks
grok-4-fast-non-reasoning	2,000,000 (2M)	\$0.20	\$0.50	—	Non-reasoning fast tasks
grok-code-fast-1	256K	\$0.20	\$1.50	\$0.02	Agentic coding
grok-3 (legacy)	131,072	\$3.00	\$15.00	—	Legacy
grok-3-mini (legacy)	131,072	\$0.30	\$0.50	—	Legacy, lightweight

Key Coding Model: grok-code-fast-1

Released August 2025, this is xAI’s purpose-built coding model:

- **SWE-Bench Verified:** 70.8% (using xAI’s internal harness)
- **Coding Accuracy:** 93.0%
- **Instruction Following:** 75.0%
- **Reliability:** 100% across seven benchmarks
- **Speed:** Up to 160 tokens/second
- **Cache hit rates:** Above 90% in partner workflows
- **Tool mastery:** Trained on grep, terminal commands, and file editing patterns
- **Language strengths:** TypeScript, Python, Java, Rust, C++, Go

Grok 4.20 Multi-Agent System

Released February 17, 2026, Grok 4.20 deploys a 4-agent system:

Agent	Role	Specialty
Grok (Captain)	Coordinator	Orchestrates other agents, synthesizes final answer

Agent	Role	Specialty
Harper	Research	Real-time search, X firehose (~68M English tweets/day), evidence gathering
Benjamin	Logic & Coding	Step-by-step reasoning, code execution, mathematical proofs
Lucas	Creative & Balance	Divergent thinking, blind-spot detection, UX optimization

For coding tasks: Benjamin handles logic and code structure, Harper checks documentation and APIs, Lucas optimizes readability and catches edge cases. The internal peer-review loop reduced hallucination rates from ~12% to 4.2% (65% reduction).

Open Source Models

Model	Parameters	License	Notes
Grok-1	314B (MoE, 8 experts)	Apache 2.0	Base model, Oct 2023 checkpoint. 8K context.
Grok-2	~500GB checkpoint	Open weights	Requires 8 GPUs with 40GB+ VRAM each
Grok-2.5	Open weights	Open source	Released August 2025 on HuggingFace

Elon Musk announced Grok 3 open source to follow in 2026.

5. Tool System — Function Calling and Built-in Tools

Built-in Server-Side Tools

These run entirely on xAI's infrastructure. No developer-side execution needed.

Tool	Description	Pricing
code_execution	Sandboxed Python environment with NumPy, Pandas, Matplotlib, SciPy pre-installed	\$2.50-5.00/1K calls
web_search	General web search	\$2.50-5.00/1K calls
x_search	Real-time X/Twitter post search	\$2.50-5.00/1K calls
collections_search	RAG over uploaded documents/codebases	\$2.50/1K searches

Code Execution Sandbox

The `code_execution` tool (also called `code_interpreter`) enables: - Python code execution in a secure sandbox - Mathematical computations requiring exact answers - Data analysis with Pandas/NumPy - Visualization with Matplotlib - Multi-step calculations with intermediate results - Financial modeling and scientific computing

Limitations: Execution time constraints, memory limits for large datasets, Python only.

Client-Side Function Calling

Developers define custom tools that Grok can request:

```
# Example: Define a custom tool
```

```
tools = [{  
    "type": "function",  
    "function": {  
        "name": "get_weather",  
        "description": "Get current weather for a location",  
        "parameters": {  
            "type": "object",  
            "properties": {  
                "location": {"type": "string"}  
            },  
            "required": ["location"]  
        },  
    }  
}]
```

```
# Grok requests the call, you execute locally, return result
```

The model requests the call, the developer executes it locally, and returns the result. This enables integration with databases, APIs, and any external system.

Structured Outputs: Grok supports enforcing JSON schemas on responses via `response_format`, ensuring consistent structured data from function calls.

Collections API (RAG for Codebases)

Launched December 22, 2025, the Collections API enables end-to-end RAG:

- **Upload:** PDFs, Excel files, entire codebases
- **Indexing:** OCR and layout-aware parsing preserves structure (PDF layout, Excel hierarchy, code syntax)
- **Search modes:** Semantic search, keyword search, hybrid search (reranker model or reciprocal rank fusion)
- **Auto-reindexing:** When files change, the system efficiently reindexes
- **Codebase use case:** Upload a repository, perform hybrid search combining keyword and semantic matching to locate specific functions or patterns
- **Pricing:** Indexing/storage free for first week; retrieval at \$2.50/1K searches

6. API Access — Using Grok for Coding

Getting Started

```
# Install the official Python SDK
```

```
pip install xai-sdk
```

```
# Or use OpenAI-compatible endpoint with any HTTP client
```

```
curl https://api.x.ai/v1/chat/completions \  
-H "Authorization: Bearer $XAI_API_KEY" \  
-H "Content-Type: application/json" \  
-d '{  
    "model": "grok-code-fast-1",  
    "messages": [{"role": "user", "content": "Write a Python function to merge two sorted arrays"}]  
'
```

Two API Endpoints

1. Chat Completions API (/v1/chat/completions) - OpenAI-compatible. Works with any tool expecting an OpenAI base URL. - Set `base_url` to `https://api.x.ai/v1` and use your xAI API key. - Supports function calling, structured outputs.

2. Responses API (/v1/responses) - Newer endpoint with richer tool support. - Server-side tools: `web_search`, `x_search`, `code_interpreter`, `collections_search`. - Mixing server-side and client-side tools in a single request. - Supports batch processing (including image/video generation in batch).

Official Python SDK

```
import xai_sdk

# Synchronous client
client = xai_sdk.Client(api_key="your-key")
response = client.chat.completions.create(
    model="grok-code-fast-1",
    messages=[{"role": "user", "content": "Refactor this function..."}]
)

# Async client
async_client = xai_sdk.AsyncClient(api_key="your-key")
```

SDK Details: - gRPC-based for high performance - Built-in retry with exponential backoff on UNAVAILABLE errors - Supports text generation, image generation/understanding, video generation, tool calling, structured outputs - Python 3.10+

Third-Party SDK Integrations

- **Vercel AI SDK:** @ai-sdk/xai provider
- **LangChain:** Via OpenAI-compatible endpoint
- **Instructor:** For structured outputs with Grok
- **Promptfoo:** xAI provider for evaluation

Batch API

The Batch API supports: - Chat completions in batch - Image generation, editing, and video generation in batch - Both server-side and client-side function tools in batch requests

7. IDE Integration — Editor Plugins

Official Support (via OpenAI-compatible API)

xAI's OpenAI-compatible API means grok-code-fast-1 works in any editor that supports custom OpenAI endpoints:

Editor	Integration Method	Status
GitHub Copilot	Native model selection	Supported (was free at launch)
Cursor	Custom API base URL (https://api.x.ai/v1)	Supported
Cline	API key + model selection in settings	Supported
Windsurf	Native (Pro and Teams users)	Supported
Kilo Code	API integration	Supported
Roo Code	API integration	Supported
opencode	API integration	Supported

Cursor Setup

```
Settings > Models > Add Model
- API Key: [your xAI key]
- Override OpenAI Base URL: https://api.x.ai/v1
- Model name: grok-code-fast-1
```

Cline Setup

1. Install Cline from VS Code marketplace
2. Open Cline, click “Use your own API key”
3. Save your xAI API key
4. Select `grok-code-fast-1` in Cline settings > API Configuration

Community VS Code Extensions

Extension	Description
Simply Grok for VSCode	Send source files to Grok and ask questions
Grok AI Integration	Direct Grok integration in VS Code
vscode-grok (eriktodx)	Query Grok about entire workspace, files, functions, or selected code

GitHub Copilot Chat

Grok is accessible through GitHub Copilot Chat, meaning developers can switch to Grok without installing a separate tool.

8. Agent Capabilities — Autonomous Coding

Current Autonomous Capabilities

grok-code-fast-1 (via IDE agents like Cline/Cursor): - Autonomous code generation, testing, and optimization
- Multi-step workflow execution without constant user input - Tool mastery: grep, terminal commands, file editing - Cache hit rates above 90% in agentic workflows - Adapts to multi-step workflows

Grok 4.20 Multi-Agent (via web UI): - 4 agents (Grok, Harper, Benjamin, Lucas) working in parallel - Benjamin handles code logic, structure, and mathematical proofs - Harper verifies against documentation - Lucas catches edge cases and optimizes readability - Peer-review loop reduces hallucination by 65%

Grok Build (coming — waitlist): - 8 parallel autonomous coding agents - Arena Mode: agents compete, outputs ranked algorithmically - Autonomous planning, searching, and code generation - Full local execution with auditable actions

What Is Missing vs. Claude Code

Capability	Claude Code	Grok (current)
Terminal-native CLI agent	Yes (GA)	No (Grok Build waitlist)
Git-aware operations	Yes (deep integration)	No native tool
Background agents	Yes (up to 5+)	Grok Build: 8 parallel (waitlist)
Hook system (17 events)	Yes	No
CLAUDE.md project context	Yes	No equivalent
Worktree isolation	Yes	No
MCP server integration	Yes	No
Auto-memory	Yes	No
Session resume	Yes	No

9. Unique Features — What Grok Does Differently

9.1 Real-Time X/Twitter Data

Grok has exclusive access to X's firehose (~68 million English tweets/day). This is unique among coding assistants:

- Search real-time developer discussions about libraries, bugs, and workarounds
- Access to trending tech discussions and announcements
- `x_search` tool available via API for programmatic access

9.2 2-Million Token Context Window

Grok 4.1 Fast offers a 2,000,000-token context window — the largest among frontier models. For coding:

- Ingest entire large codebases in a single prompt
- No chunking or RAG needed for many projects
- Particularly useful for monorepo analysis

9.3 Multi-Agent Architecture (Grok 4.20)

The 4-agent debate system (Grok, Harper, Benjamin, Lucas) is unique:

- Agents approach problems from different domains
- Internal peer review before final answer
- 65% hallucination reduction vs. single-agent approach
- Not a user-facing framework — it runs automatically on complex queries

9.4 Arena Mode (Grok Build — Coming)

No other coding tool offers this:

- 8 agents produce solutions simultaneously
- Automated evaluation ranks outputs algorithmically
- Developer reviews pre-ranked solutions rather than raw outputs
- Combines the “best of N” approach with human oversight

9.5 Collections API for Codebase RAG

Upload entire codebases and get hybrid search (semantic + keyword):

- Layout-aware parsing preserves code structure
- Auto-reindexing when files change
- Combine with Grok's tool calling for codebase-aware AI

9.6 Aggressive Pricing

grok-code-fast-1 at \$0.20/M input and \$1.50/M output is significantly cheaper than:

- Claude Opus 4.6: \$15/M input, \$75/M output
- GPT-5.3 Codex: ~\$5/M input, ~\$15/M output
- Even Claude Sonnet 4: \$3/M input, \$15/M output

9.7 Speed

grok-code-fast-1 at up to 160 tokens/second outperforms many competitors in raw speed for code generation.

10. Limitations — Honest Assessment

10.1 No Shipping CLI Agent (Yet)

The most critical gap. As of March 2026:

- **Claude Code:** GA, polished, deeply integrated with Git, MCP, hooks, worktrees, background agents, memory, session resume.
- **Codex CLI:** GA, sandbox-first, terminal-native.
- **Grok Build:** Still on waitlist. Not generally available.

The community Grok CLI exists but is third-party, has had maintenance issues, and lacks the polish of official tools.

10.2 Code Quality Consistency

Multiple independent reviewers report:

- Grok generates code that is fast but less consistent than Claude or GPT for complex tasks
- More syntax errors and logical flaws in complex refactors
- Hallucinated library names occasionally
- Claude is preferred for detailed line-by-line code commentary and fixing logic errors

10.3 No Vision for Code (Partial)

Claude can read screenshots and wireframes for UI debugging. Grok’s vision capabilities for coding are limited — a significant bottleneck for teams doing UI work.

10.4 No Git Integration

No native Git-aware operations. Claude Code understands Git history, branches, diffs, and can commit/push. Grok has no equivalent.

10.5 No Project Context System

Claude Code has CLAUDE.md files for project-specific instructions, auto-memory that persists across sessions, and session resume. Grok has no equivalent persistent project context.

10.6 No Hook System

Claude Code’s 17 programmable hook events (pre-tool, post-tool, notification, etc.) enable CI/CD integration, security scanning, and custom workflows. Grok has nothing comparable.

10.7 No MCP Integration

Claude Code’s Model Context Protocol enables connecting to databases, APIs, and tools natively. Grok has no equivalent protocol.

10.8 Ecosystem Fragmentation

Grok’s coding story is spread across: - Grok Studio (web, prototyping) - Grok Build (CLI, waitlist) - grok-code-fast-1 (model, in third-party editors) - Grok 4.20 multi-agent (web UI) - Community CLI (third-party) - Collections API (RAG)

There is no single unified experience. Claude Code is one tool that does it all.

10.9 Speed vs. Depth Trade-off

grok-code-fast-1 is fast but often sacrifices the “thinking aloud” that Claude provides: - Claude walks through reasoning, context, and edge cases — valuable for complex refactors - Grok gives ready-to-run code faster but with less explanation - For learning and debugging, Claude’s verbosity is an advantage

11. Roadmap — What Is Coming

Confirmed / In Development

Feature	Expected	Source
Grok Build GA	Q1-Q2 2026	Waitlist active at grokai.build
Arena Mode	With Grok Build	Code traces found Feb 2026
Grok 5	Q2 2026 (delayed from Q1)	6 trillion parameters, Colossus 2 supercluster
Grok 3 Open Source	2026	Announced by Elon Musk
GitHub integration in Grok Build	With Grok Build GA	Found in code analysis
Collaboration features	With Grok Build GA	Share and Comments in code traces

Grok 5 Details

Originally scheduled for Q1 2026, now expected Q2 2026: - 6 trillion parameters (largest publicly announced AI model) - Trained on xAI’s Colossus 2 supercluster - Expected to significantly improve coding capabilities - Still in training as of March 2026

Grok 4.20 Evolution

Grok 4.20 Beta 2 (March 3, 2026) continues to refine the multi-agent system. The 4-agent architecture (Grok, Harper, Benjamin, Lucas) is expected to improve with: - Better inter-agent coordination - Reduced latency in agent debate cycles - Improved coding specialization for Benjamin

12. Comparison Matrix

Grok vs. Claude Code vs. Codex CLI (March 2026)

Dimension	Claude Code	Codex CLI	Grok (Best Available)
Primary Interface	Terminal CLI	Terminal CLI	Web (Studio) + IDE plugins + API
CLI Agent	GA, polished	GA	Waitlist (Grok Build)
Default Model	Opus 4.6 (200K ctx)	GPT-5.4 (1M ctx)	grok-code-fast-1 (256K) / grok-4.1-fast (2M ctx)
SWE-Bench	80.8% (Opus 4.6)	75.1% (GPT-5.4)	70.8% (grok-code-fast-1)
Multi-Agent	Background agents (5+)	Single agent	8 parallel (Grok Build, waitlist) / 4-agent debate (4.20)
Git Integration	Deep (commit, diff, branch)	Basic	None
Project Context	CLAUDE.md + auto-memory	AGENTS.md	None
Hook System	17 events	Sandboxed	None
MCP Integration	Yes	No	No
Session Resume	Yes	No	No
IDE Support	Native (VS Code, JetBrains)	VS Code	Via OpenAI-compatible API (Cursor, Cline, Windsurf, Copilot)
Real-Time Data	No	No	X firehose (unique)
Pricing (coding model)	\$15/\$75 per M (Opus)	~\$5/\$15 per M (GPT-5.3)	\$0.20/\$1.50 per M (code-fast-1)
Speed	Moderate	Fast	Very fast (160 tok/s)
Open Source	No	Yes (Apache 2.0)	Community CLI; Grok-1/2/2.5 weights open
Local Execution	Yes	Yes (sandboxed)	Yes (Grok Build) / No (Studio)
Code Privacy	Code stays local	Code stays local	Local (Grok Build) / Cloud (Studio, API)

Bottom Line

- **Choose Claude Code** if you want the most mature, integrated terminal coding experience with deep Git, MCP, hooks, and project context.
- **Choose Codex CLI** if you want strong terminal automation with kernel-level sandboxing.
- **Choose Grok** if you need the cheapest API for coding, the fastest inference, the largest context window (2M tokens), real-time X data, or want to wait for Grok Build’s parallel agent architecture.

Appendix: Key URLs

Resource	URL
xAI API Console	https://console.x.ai
xAI Docs	https://docs.x.ai
Models & Pricing	https://docs.x.ai/developers/models
Tool Calling Docs	https://docs.x.ai/docs/guides/tools/overview
Function Calling Guide	https://docs.x.ai/docs/guides/function-calling
Code Execution Tool	https://docs.x.ai/docs/guides/tools/code-execution-tool
Collections API	https://docs.x.ai/docs/guides/tools/collections-search-tool
IDE Setup Guide	https://docs.x.ai/developers/advanced-api-usage/use-with-code-editors
Grok Build Waitlist	https://grokai.build
Grok Studio	https://grokai.studio
Python SDK (GitHub)	https://github.com/xai-org/xai-sdk-python
Python SDK (PyPI)	https://pypi.org/project/xai-sdk/
Community Grok CLI	https://github.com/superagent-ai/grok-cli
Open Source Grok-1	https://github.com/xai-org/grok-1
grok-code-fast-1 Announcement	https://x.ai/news/grok-code-fast-1

This document reflects the state of xAI's coding tools as of March 25, 2026. Grok's coding ecosystem is evolving rapidly — Grok Build's general availability could significantly change this assessment.